The Box Model & the Cascade

Sizing Things Up, Winning the Cascade — In-Class Discovery Worksheet

Name: _____ Section: _____ Date: _____

How to use this sheet: Keep your **browser**, **VS Code**, and **DevTools** open beside you. For every task, **attempt and write your answer *before* we discuss it** — your first guesses matter more than being right.

Part 1 — Discovering the Box Model

TRY THIS · VS CODE

Create a fresh `boxmodel.html` with the usual boilerplate. In the `<body>`, add a `<p>` with some text and `background-color: lightblue;` — nothing else.

Q1. Is the color touching the text exactly, or is there space around it? Where do you think that space is coming from?

TRY THIS · BROWSER

Right-click your paragraph and choose **Inspect**. Find the box diagram — a set of nested, labeled rectangles — in the **Elements** tab's Styles or Computed panel.

Q2. What do you see in that diagram? Any guess what the nested rectangles and numbers represent?

TRY THIS · VS CODE

Add `padding: 20px;` to your `<p>`. Then inspect it again.

Q3. What changed — both on the page and in the DevTools diagram? Is the extra space inside the colored background, or outside it?

TRY THIS · VS CODE

Add `border: 5px solid black;` to the same `<p>`. Then inspect again.

Q4. Where did the border land — before the padding, or after it? What's it sitting between?

TRY THIS · VS CODE

Add `margin: 20px;`. Add a second, identical `<p>` below it so you can see the space between two elements.

Q5. Does margin have background color on it, the way padding did?

Q6. Now that you've seen all four layers together in DevTools, describe the box model in your own words, from the inside out — content, padding, border, margin — and what each layer does.

Part 2 — Controlling Sides, Centering, and a Gotcha

TRY THIS · VS CODE

Instead of `padding: 20px;`, try setting each side separately: `padding-top: 10px; padding-right: 40px; padding-bottom: 10px; padding-left: 5px;`. Then inspect.

Q7. Was that intuitive, or surprising? Explain your reasoning in a sentence or two.

TRY THIS · VS CODE

Replace those four lines with one: `padding: 10px 40px 10px 5px;` and compare to before. Then try `margin: 10px 40px;` — only two values — on the same element.

Q8. For each shorthand below, write which side (top, right, bottom, left) each value applies to.

```
padding: 10px;                  ____
padding: 10px 20px;             ________  ________
padding: 10px 20px 30px;        ____  ________  ____
padding: 10px 20px 30px 40px;   ____  ____  ____  ____
```

TRY THIS · VS CODE

Give your `<p>` a fixed `width: 300px;` and `margin: 0 auto;`, inside a `<body>` with no other width restrictions.

Q9. What happened to the box? Any guess why `auto` on left/right margin has this centering effect, but `auto` wouldn't do the same thing for top/bottom margin?

TRY THIS · VS CODE

Add two `<p>` tags, stacked — the first with `margin-bottom: 30px;`, the second with `margin-top: 20px;`. Inspect the actual gap between them in DevTools.

Q10. Is the gap 50px (30 + 20), or something else? Does this same behavior happen with padding, or with left/right margins? Take a guess, then test it — what does this tell you about how top/bottom margins behave between stacked elements?

Part 3 — box-sizing: content-box vs border-box

TRY THIS · VS CODE

Give your `<p>` a fixed `width: 300px;` and `background-color: lightblue;` only — no padding, no border yet. Check the Computed width in DevTools.

Q11. What's the Computed width showing? Does it match the 300px you set?

TRY THIS · VS CODE

Now add `padding: 40px;` and `border: 5px solid black;` to the same element. Then check the Computed width again.

Q12. Is the box still 300px wide, or did it grow? If it grew, can you connect that number back to your padding and border values? Based on this, what do you think `width` actually applies to — the content only, or the whole box?

This default behavior — where `width` sets only the content, and padding/border are added on top of it — is called **content-box**. It's the default for every element unless you say otherwise.

TRY THIS · VS CODE

Add one line to the same element: `box-sizing: border-box;`. Keep `width: 300px;`, `padding: 40px;`, and `border: 5px solid black;` unchanged. Then check the Computed width.

Q13. What's the Computed width now? What do you think `box-sizing: border-box` told the browser to do differently with your padding and border?

TRY THIS · VS CODE

Duplicate your `<p>` so you have two — same width, padding, and border, but one with `content-box` (or nothing, since that's the default) and the other with `border-box`.

Q14. Which one actually stays exactly 300px wide on your screen? If you were laying out a page with lots of boxes that all need to line up, which would be easier to work with?

Part 4 — The Universal Selector

TRY THIS · VS CODE

Make sure your page has at least 3-4 different elements — say an `<h1>`, two `<p>`s, and a `<div>`. Give each one `margin: 20px;` and `padding: 20px;` separately, writing a full rule for every single one (`h1 { margin: 20px; padding: 20px; }`, `p { margin: 20px; padding: 20px; }`, and so on). Then delete all of that and replace it with just one rule: `* { margin: 20px; padding: 20px; }`. Inspect a few different elements, including `<body>` and `<html>`.

Q15. What was writing a separate rule for every tag like — and what would that get like on a page with 20 different kinds of elements? Once you switched to `*`, did every element get the same margin and padding — including `<body>` and `<html>`, not just the tags you'd written rules for? What do you think `*` is selecting, compared to a tag selector like `h1` or `p`?

Part 5 — Specificity: Who Wins When Rules Conflict?

TRY THIS · VS CODE

Create a fresh `specificity.html` with just `<h1>Hello</h1>` — no CSS at all, not even a `<style>` block. Open DevTools → Elements → Styles panel. Then add `* { color: green; }` to a `<style>` block.

Q16. Even with zero CSS written by you, did the `<h1>` already look a certain way — bold, big, with its own spacing? Where does DevTools say that's coming from?

That default look comes from the **user-agent stylesheet** — a base set of CSS rules every browser ships and applies before any of your own CSS runs. It's why an unstyled `<h1>` is already bold and large, why `<p>`s already have margin, and why `<a>` links are already blue and underlined. It sits at the very bottom of the cascade, so any rule you write — even a plain tag selector — overrides it.

TRY THIS · VS CODE

Add `h1 { color: blue; }` below your `*` rule, keeping both.

Q17. Which color won — green or blue? Between the universal selector and a tag selector, which one seems more powerful?

TRY THIS · VS CODE

Give your `<h1>` a `class="highlight"`. Add `.highlight { color: orange; }`, keeping the `h1 { color: blue; }` rule too.

Q18. Which color won this time? Between a tag selector and a class selector, which is more powerful?

TRY THIS · VS CODE

Give the same `<h1>` an `id="special"` too. Add `#special { color: teal; }`, keeping the class rule.

Q19. Which color won — orange or teal? Between class and id, which is more powerful?

TRY THIS · VS CODE

Add `style="color: red;"` directly on the `<h1>` tag, keeping the `#special` rule. Then change `#special { color: teal; }` to `#special { color: teal !important; }`, keeping the inline style.

Q20. Which color won with the inline style — teal or red? Where does inline CSS rank compared to id, class, and tag? Once you added `!important`, which color won — and what do you think `!important` does to a rule's priority?

Q21. Based on everything you just tested, put these in order from weakest to strongest: tag selector, id selector, class selector, inline style, universal selector, `!important`, user-agent stylesheet.

```
1 _______________  2 _______________  3 _______________  4 _______________
5 _______________  6 _______________  7 _______________
```

Part 6 — Units: px, %, vw, vh

TRY THIS · VS CODE

Create a fresh `units.html`. Add a `<div>` with `width: 300px; height: 100px; background-color: lightblue;`. Resize your browser window while watching the box.

Q22. What do you think `px` means here? Does the box change size as you resize the window?

TRY THIS · VS CODE

Change the `<div>`'s width to `width: 50%;`, keeping it inside a plain `<body>` with no other width set. Resize your browser window while watching the box.

Q23. What happened to the box as you resized the window? What do you think `%` is measuring relative to?

TRY THIS · VS CODE

Keep the `<div>` at `width: 50%;`, still sitting directly inside `<body>` (no wrapper element). Now give `<body>` itself a fixed `width: 600px;`. Resize your browser window while watching the box.

Q24. Now that `<body>` has a fixed width instead of stretching with the window, did the div's size change from before — and does it still change as you resize the window? What does this tell you about what `%` is actually measured relative to?

TRY THIS · VS CODE

Keeping `<body>` fixed at `width: 600px;`, now try `width: 50vw;` on the `<div>` instead of `50%`. Resize the window again.

Q25. Does this look similar to the `%` result, or different — even with `<body>`'s width now fixed? Any guess what the "vw" in `vw` stands for?

TRY THIS · VS CODE

Change `height: 100px;` to `height: 50vh;`. Resize your browser window's height (or use DevTools' device toolbar) and watch the box.

Q26. What changed this time? What do you think "vh" stands for, and what's it measuring relative to? Overall, which of these four units change automatically when the window is resized, and which one doesn't?